

Workflow Guide

NU-CCIT Capstone Starter

Contents

Introduction	1
1. Git Branching & PR Convention	2
1.1 Branch naming	2
1.2 Main branches	2
1.3 Commit messages	2
1.4 Opening a pull request	3
1.5 Resolving merge conflicts	3
1.6 Merging to main	3
2. Adding a New Model-Controller-Route Trio (Backend)	4
2.1 The Model	4
2.2 The Controller	4
2.3 The Route	5
2.4 Wiring it into the app	6
2.5 Checklist for a new resource	6
3. API Testing Guide (Postman / Thunder Client)	7
3.1 Choosing a tool	7
3.2 Setting up an environment	7
3.3 Building a collection	7
3.4 Testing auth-protected routes	7
3.5 Sharing the collection with the team	8
3.6 Quick checklist before merging a backend PR	8

Introduction

This guide covers team workflow conventions for a multi-layer capstone repo (backend, frontend, mobile, infra, novelty, manuscript). With several contributors touching different layers at once, a shared convention keeps everyone from stepping on each other.

1. Git Branching & PR Convention

1.1 Branch naming

Use a `<type>/<layer>-<short-description>` format so it's obvious what a branch touches at a glance:

```
feature/backend-user-auth
feature/frontend-login-page
feature/mobile-attendance-scanner
fix/backend-jwt-expiry-bug
chore/infra-docker-compose-update
docs/manuscript-chapter-3-draft
```

Common `<type>` prefixes:

Prefix	Use for
feature/	New functionality
fix/	Bug fixes
chore/	Config, tooling, dependency bumps
docs/	Documentation, manuscript edits
refactor/	Code restructuring with no behavior change

1.2 Main branches

Branch	Purpose
main	Always deployable/demo-ready. Protected — no direct pushes.
develop	Integration branch where feature branches merge before main.

Create feature branches off develop, not main:

```
git checkout develop
git pull origin develop
git checkout -b feature/backend-user-auth
```

1.3 Commit messages

Keep commits small and descriptive. A simple convention that works well:

`<type>: <short summary>`

```
feat: add JWT refresh token endpoint
fix: correct off-by-one error in pagination
docs: update .env reference in guides
```

Avoid vague commits like `update stuff` or `fix bug` — a future teammate (or you, in three weeks) won't know what changed without opening the diff.

1.4 Opening a pull request

1. Push your branch and open a PR **into develop**, not main.
2. Fill in what changed and why — a one-paragraph summary is enough for most PRs.
3. Link the related task/issue if your team is tracking tasks somewhere (Trello, GitHub Issues, etc.).
4. Request review from at least one teammate working on the same or an adjacent layer.
5. Don't merge your own PR without a review, even under deadline pressure — a second pair of eyes catches merge conflicts and layer-breaking changes early.

1.5 Resolving merge conflicts

Since backend/frontend/mobile are separate folders, most conflicts will be in shared files (`README.md`, root configs) rather than in each other's code. When a conflict does happen:

```
git checkout feature/your-branch
git pull origin develop
# resolve conflicts in the flagged files
git add .
git commit
git push
```

If you're unsure which version of a conflicting line is correct, ask the teammate who wrote the other side rather than guessing.

1.6 Merging to main

Only merge develop into main at stable checkpoints — before a demo, a defense, or a deployment. Tag these merges so you can roll back easily if something breaks:

```
git tag -a v1.0-demo -m "Midterm demo checkpoint"
git push origin v1.0-demo
```

2. Adding a New Model-Controller-Route Trio (Backend)

This walks through adding a new resource — using Task as the example — following the layered MVC pattern in backend/.

2.1 The Model

Create backend/models/Task.js:

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    description: { type: String, default: '' },
    status: {
      type: String,
      enum: ['pending', 'in-progress', 'done'],
      default: 'pending',
    },
    assignedTo: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Task', taskSchema);
```

2.2 The Controller

Create backend/controllers/taskController.js. Keep route handlers thin — they call the model, handle errors, and shape the response:

```
const Task = require('../models/Task');

exports.getAllTasks = async (req, res) => {
  try {
    const tasks = await Task.find().populate('assignedTo', 'name email');
    res.status(200).json(tasks);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

exports.getTaskById = async (req, res) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) return res.status(404).json({ message: 'Task not found' });
    res.status(200).json(task);
  }
};
```

```

    } catch (err) {
      res.status(500).json({ message: err.message });
    }
  };

exports.createTask = async (req, res) => {
  try {
    const task = await Task.create(req.body);
    res.status(201).json(task);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
};

exports.updateTask = async (req, res) => {
  try {
    const task = await Task.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true,
    });
    if (!task) return res.status(404).json({ message: 'Task not found' });
    res.status(200).json(task);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
};

exports.deleteTask = async (req, res) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);
    if (!task) return res.status(404).json({ message: 'Task not found' });
    res.status(200).json({ message: 'Task deleted' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

```

2.3 The Route

Create backend/routes/taskRoutes.js:

```

const express = require('express');
const router = express.Router();
const taskController = require('../controllers/taskController');

router.get('/', taskController.getAllTasks);
router.get('/:id', taskController.getTaskById);
router.post('/', taskController.createTask);

```

```
router.put('/:id', taskController.updateTask);
router.delete('/:id', taskController.deleteTask);

module.exports = router;
```

2.4 Wiring it into the app

In your main server file (e.g. `backend/app.js` or `backend/server.js`), mount the new route:

```
const taskRoutes = require('./routes/taskRoutes');
app.use('/api/tasks', taskRoutes);
```

2.5 Checklist for a new resource

- ☐ Model created with schema validation (required, enum, etc. where relevant)
- ☐ Controller handles all CRUD operations with try/catch error handling
- ☐ Route file maps HTTP verbs to controller functions
- ☐ Route mounted in the main app file under a sensible `/api/...` prefix
- ☐ Tested with Postman/Thunder Client (see Section 3) before merging

3. API Testing Guide (Postman / Thunder Client)

3.1 Choosing a tool

- **Postman** — standalone app, best if you want shared team collections synced to a workspace.
- **Thunder Client** — VS Code extension, lighter weight, good if you want testing without leaving your editor.

Either works fine for this project; pick whichever your team prefers and stick with one so collections can be shared.

3.2 Setting up an environment

Create an environment (Postman: **Environments** tab; Thunder Client: **Env** tab) with a variable for your API base URL, so you're not hardcoding it into every request:

```
baseUrl = http://localhost:3000/api
```

Switch this to your Atlas-backed or deployed URL later without editing every request.

3.3 Building a collection

Group requests by resource, mirroring your route files. For the Task example above:

Request	Method	URL
Get all tasks	GET	{{baseUrl}}/tasks
Get task by ID	GET	{{baseUrl}}/tasks/:id
Create task	POST	{{baseUrl}}/tasks
Update task	PUT	{{baseUrl}}/tasks/:id
Delete task	DELETE	{{baseUrl}}/tasks/:id

For POST/PUT requests, set the body type to **JSON** and provide a sample payload:

```
{
  "title": "Finish backend routes",
  "description": "Wire up task CRUD endpoints",
  "status": "in-progress"
}
```

3.4 Testing auth-protected routes

If a route requires a JWT, first hit your login endpoint to get a token, then attach it to subsequent requests:

- **Postman:** Authorization tab → type **Bearer Token** → paste the token, or set it as an environment variable and reference it as {{token}}.
- **Thunder Client:** same idea under the **Auth** tab of a request.

A quick pattern: save the token from the login response into an environment variable automatically using a post-request script, so you don't copy-paste it manually every time.

3.5 Sharing the collection with the team

- **Postman:** Export the collection (. . . menu → **Export**) as a JSON file and commit it to backend/postman/ (or wherever your team keeps it), or use a shared Postman workspace if everyone has accounts.
- **Thunder Client:** Export via the collection's . . . menu → **Export**, and commit the resulting JSON file the same way.

Keeping the collection in the repo means new routes get test requests added in the same PR that adds the route — nobody has to guess how to call a new endpoint.

3.6 Quick checklist before merging a backend PR

- ☐ Every new endpoint has a request in the shared collection
- ☐ Happy path tested (valid input, expected status code)
- ☐ At least one error case tested (invalid input, missing auth, not-found ID)
- ☐ Collection re-exported and committed if requests were added or changed